

IDMA 2024 Problem Set 1

Lauritz Tørholm Jørgensen, fqm630@alumni.ku.dk

May 2026

1

1a

The algorithm returns an array B of length n .

For each index i , the entry $B[i]$ is the number of other indices j such that $A[j]$ has the same value as $A[i]$. The condition $i \neq j$ means that the algorithm does not count the element $A[i]$ itself.

Thus,

$$B[i] = |\{j \mid 1 \leq j \leq n, j \neq i, A[j] = A[i]\}|.$$

So if a value occurs k times in A , then every position containing that value gets value $k - 1$ in B .

For example, if

$$A = [4, 2, 4, 4, 1],$$

then the value 4 occurs three times, so the three entries in B corresponding to the value 4 are equal to 2. The values 2 and 1 occur only once, so their entries in B are equal to 0. Hence the algorithm returns

$$B = [2, 0, 2, 2, 0].$$

1b

The first loop runs n times and initializes the array B .

The second part of the algorithm consists of two nested loops. The outer loop runs n times, and for each value of i , the inner loop also runs n times. Hence the comparison

$$i \neq j \text{ and } A[i] = A[j]$$

is checked n^2 times.

Each iteration takes constant time. Therefore the two nested loops take time $\Theta(n^2)$. The initialization loop takes time $\Theta(n)$, which is smaller, so the total running time is

$$\Theta(n^2).$$

1c

Now suppose that all entries in A are integers between 1 and n . Then we can use an extra array $C[1 \dots n]$ to count how many times each value occurs.

The improved algorithm is:

```

for  $i := 1$  upto  $n$ 
     $C[i] := 0$ 
for  $i := 1$  upto  $n$ 
     $C[A[i]] := C[A[i]] + 1$ 
for  $i := 1$  upto  $n$ 
     $B[i] := C[A[i]] - 1$ 
return  $B$ 

```

The array C stores the number of occurrences of each possible value. After the second loop, $C[x]$ is the number of indices i such that $A[i] = x$.

For a fixed index i , the value $A[i]$ occurs $C[A[i]]$ times in total. One of these occurrences is the element at index i itself, so the number of other equal elements is

$$C[A[i]] - 1.$$

This is exactly the value that should be stored in $B[i]$.

The running time is $\Theta(n)$, since the algorithm consists of three loops that each run n times. This improves the original running time from $\Theta(n^2)$ to $\Theta(n)$.

The algorithm is asymptotically optimal. Any correct algorithm must output the whole array B , which has n entries. Therefore any algorithm needs at least $\Omega(n)$ time. Since the new algorithm runs in $\Theta(n)$ time, it is optimal up to constant factors.

2

2a

The algorithm checks whether the array is sorted in nondecreasing order.

For a call $\text{CHECK}(A, lo, hi)$, the algorithm first chooses the middle index

$$mid = \left\lfloor \frac{lo + hi}{2} \right\rfloor.$$

It then checks two things. First, every element to the left of mid must be at most $A[mid]$. Second, every element to the right of mid must be at least $A[mid]$.

If these checks succeed, the algorithm recursively checks the left part

$$A[lo \dots mid - 1]$$

and the right part

$$A[mid + 1 \dots hi].$$

Therefore, when `CHECK( $A, 1, A.size$ )` returns `TRUE`, the whole array A is sorted in nondecreasing order. That is, for all indices $i < j$,

$$A[i] \leq A[j].$$

If the algorithm finds an element on the left side that is greater than $A[mid]$, or an element on the right side that is smaller than $A[mid]$, it returns `FALSE`.

2b

Let n be the size of the array.

In one call on a subarray of size n , the algorithm compares the middle element with the other elements in the subarray. This takes $\Theta(n)$ time in the worst case.

The algorithm then recursively calls itself on the left and right halves of the subarray. These two parts have sizes approximately $n/2$. Hence the running time satisfies the recurrence

$$T(n) = 2T(n/2) + \Theta(n).$$

By the master theorem, this gives

$$T(n) = \Theta(n \log n).$$

So the worst-case running time of the algorithm is

$$\Theta(n \log n).$$

2c

Yes, the algorithm can be improved. Since the function only checks whether the array is sorted, it is enough to compare each element with the next one.

An improved algorithm is:

```

for  $i := 1$  upto  $A.size - 1$ 
    if  $A[i] > A[i + 1]$ 
        return FALSE
return TRUE

```

This algorithm returns FALSE if it finds two neighboring elements that are in the wrong order. If this never happens, then

$$A[1] \leq A[2] \leq \dots \leq A[n],$$

so the array is sorted.

The running time is

$$\Theta(n),$$

since the algorithm makes one left-to-right pass through the array.

This is asymptotically faster than the original algorithm, which runs in $\Theta(n \log n)$ time.

The improved algorithm is asymptotically optimal. In the worst case, a correct algorithm must inspect the array entries to decide whether the array is sorted. If an algorithm skipped some entry, changing that entry could make the array unsorted without the algorithm noticing. Thus any correct algorithm needs at least $\Omega(n)$ time in the worst case.

Since the improved algorithm runs in $\Theta(n)$ time, it is optimal up to constant factors.

3

3a

The algorithm searches for three distinct indices i , j , and k such that

$$A[i] + A[j] + A[k] = 0.$$

The condition

$$i \neq j \text{ and } j \neq k \text{ and } i \neq k$$

ensures that the three indices are all different. The algorithm tries all ordered triples of indices until it either finds such a triple or has checked all possibilities.

If the algorithm does not return "failed", then it returns a triple

$$(i, j, k)$$

where i , j , and k are distinct indices and

$$A[i] + A[j] + A[k] = 0.$$

The triple returned is a triple of indices, not the actual values from the array.

3b

The algorithm has three nested loops. In the worst case, no valid triple exists, or the first valid triple is found very late. Then the algorithm may have to check essentially all triples of indices.

The outer loop can run n times. For each value of i , the middle loop can run n times. For each pair (i, j) , the inner loop can run n times. Thus the total number of iterations is proportional to

$$n \cdot n \cdot n = n^3.$$

Each iteration takes constant time, since it only checks a few inequalities and computes one sum. Therefore the worst-case running time is

$$\Theta(n^3).$$

3c

Yes, the algorithm can be improved. The problem can be solved by first sorting the array and then using two pointers.

Since the algorithm has to return indices from the original array, we store both the value and the original index before sorting.

```
for  $i := 1$  upto  $n$ 
     $B[i].value := A[i]$ 
     $B[i].index := i$ 
sort  $B$  by  $B[i].value$ 
for  $p := 1$  upto  $n - 2$ 
     $q := p + 1$ 
     $r := n$ 
    while  $q < r$ 
         $s := B[p].value + B[q].value + B[r].value$ 
        if  $s = 0$ 
            return  $(B[p].index, B[q].index, B[r].index)$ 
        if  $s < 0$ 
             $q := q + 1$ 
        else
             $r := r - 1$ 
return "failed"
```

The algorithm is correct because after sorting, the values are in increasing order. For a fixed position p , we look for two positions q and r such that

$$B[p].value + B[q].value + B[r].value = 0.$$

If the sum is too small, then increasing q can make the sum larger. If the sum is too large, then decreasing r can make the sum smaller. Therefore the two-pointer search does not skip a possible solution for the fixed value of p .

If some valid triple exists, then in sorted order it has some smallest position p . When the outer loop reaches that p , the two-pointer search will find the other two positions.

The sorting step takes $O(n \log n)$ time. The outer loop runs n times, and for each value of p , the two pointers together move at most n steps. Therefore the search part takes $O(n^2)$ time. The total running time is

$$O(n^2).$$

This is asymptotically faster than the original algorithm, which runs in $\Theta(n^3)$ time.

I cannot prove that this $O(n^2)$ algorithm is asymptotically optimal in the usual model of computation. I can only prove the simple lower bound $\Omega(n)$, since an algorithm must inspect the input in the worst case. A matching $\Omega(n^2)$ lower bound for the general problem is not known. This is why the problem appears again in the bonus question.

3d

This is the 3SUM problem.

I found it by searching on Google. I tried different variations of “X problem”, for example “ijk sum problem” and “three variable problem”. The search that got me there was “three variable sum problem”. In retrospect, it would probably have been easier to search for parts of the pseudocode directly.

The problem and standard solutions can be found on Wikipedia,

<https://en.wikipedia.org/wiki/3SUM>,

and on LeetCode,

<https://leetcode.com/problems/3sum/description/>.

The usual textbook-style algorithm sorts the array and then uses two pointers, giving an $O(n^2)$ algorithm. This is the algorithm used in part 3c.

The research question is whether 3SUM can be solved asymptotically faster than quadratic time. Older versions of the 3SUM conjecture said that the obvious $O(n^2)$ algorithm was optimal. This was refuted by Grønlund and Pettie, who gave subquadratic algorithms. One of their deterministic bounds was

$$O\left(\frac{n^2}{(\log n / \log \log n)^{2/3}}\right).$$

A later bound by Chan gives

$$O\left(\frac{n^2}{\log^2 n} (\log \log n)^{O(1)}\right).$$

This is the best upper bound I found for the real-number version of 3SUM.
For lower bounds, the answer depends on the model of computation. Erickson proved an

$$\Omega(n^2)$$

lower bound in a restricted linear decision-tree model. But this does not prove that every ordinary algorithm must take quadratic time. In stronger models, the exact complexity is still open. A common modern form of the 3SUM conjecture is that there is no algorithm running in

$$O(n^{2-\varepsilon})$$

time for any fixed $\varepsilon > 0$.

So for part 3c, I can give an $O(n^2)$ algorithm, but I cannot prove it optimal in the general setting.

Note: except for this specific sub-question and the very last paragraph in 3c, I have not used the information found on the Internet.